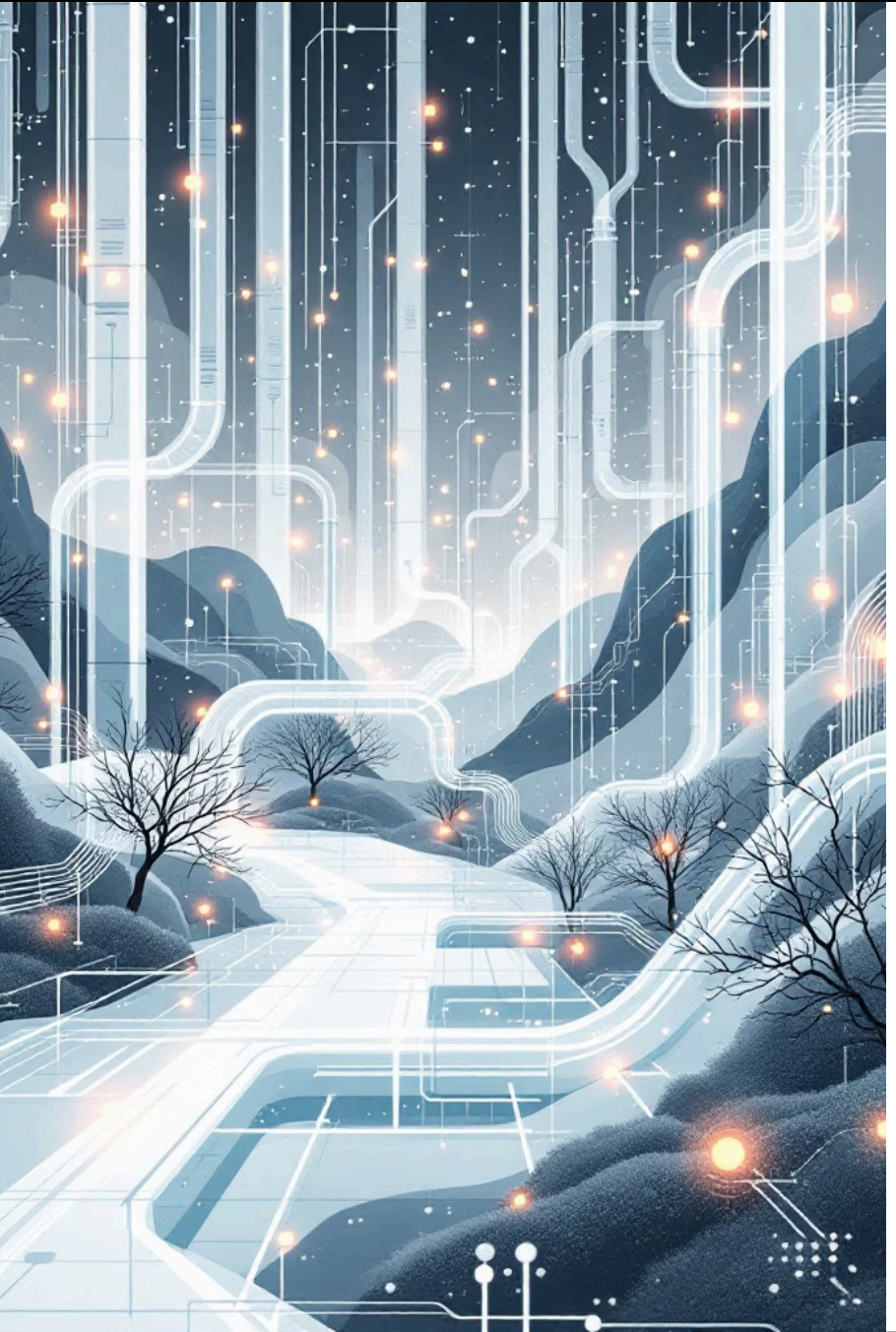


Pós-Graduação em Aprendizagem Automática Aplicada

# Intelligent Models Engineering

## Module 2 - Design (Data & Model Engineering)

António Raimundo
Professor Auxiliar no Iscte-Sintra

Made with GAMMA

# The Reproducibility Crisis: Hidden Debt in ML Systems

## Why Git Alone Is Insufficient

Git tracks **code state** with precision- but an ML experiment depends on three tightly coupled axes. Versioning only one axis yields a fundamentally non-reproducible system.

Rerunning the same commit 6 months later with different data silently produces different models, with no error, no warning, no audit trail.

The **Reproducibility Triangle**: all three axes must be pinned simultaneously for a run to be deterministically replayable.

### 🧬 Code

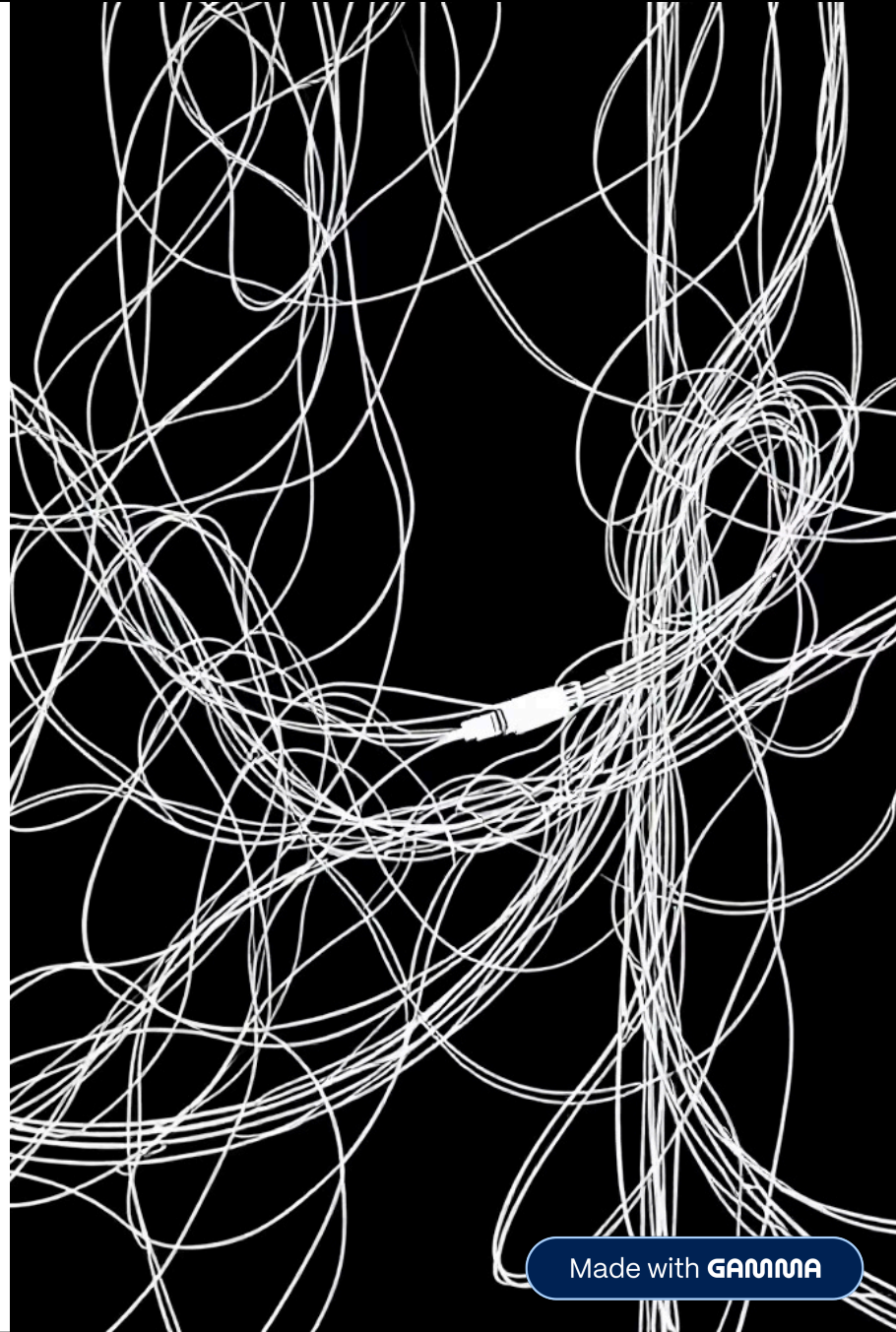
Tracked by Git — branch, commit SHA, diff history

### 🗄️ Data

Versioned by DVC — content-addressable, immutable snapshots

### ⚙️ Config

Logged by MLflow — hyperparameters, runtime environment, seeds



Made with GAMMA

# DVC as the Data Backbone

DVC decouples large binary assets from Git by replacing them with lightweight **.dvc metafile pointer files**: Committing metadata, not the data itself.

## Content-Addressable Storage via MD5

Every tracked file receives an MD5 hash computed from its **content**, not its filename or path. Identical content always yields an identical hash, enabling deterministic cache lookups and collision-free deduplication across remote storage backends (S3, Azure Blob, GCS, SSH).

> **Immutability guarantee:** a hash in the DVC cache is write-once. The underlying snapshot can never be silently overwritten.

## Anatomy of a .dvc Pointer File

```
outs:
- md5: a1b2c3d4e5f6...
  size: 1048576000
  path: data/train.csv
  cache: true
  persist: false
```

This YAML artifact is committed to Git. The actual dataset lives in `.dvc/cache/` or a configured remote, fully outside the repository tree.

Made with GAMMA

# MLflow as the Flight Recorder

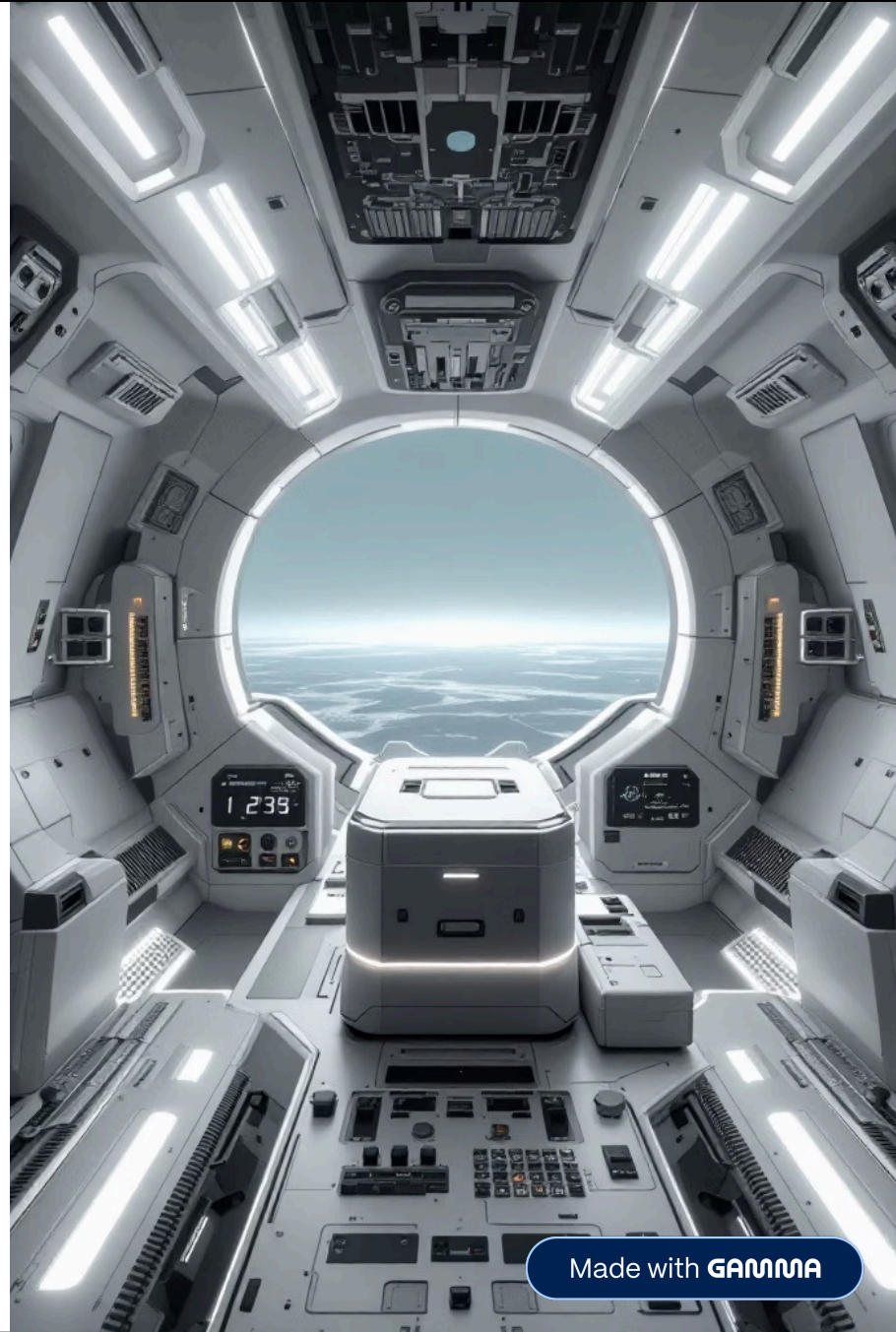
## Experiments & Runs
## The Immutable Ledger

An **Experiment** is a named logical grouping; a **Run** is an atomic, timestamped execution unit capturing the complete state of one training invocation.

- **Parameters**: hyperparameters, dataset paths, random seeds
- **Metrics**: loss curves, accuracy, RMSE, elapsed times
- **Artifacts**: models, plots, performance evaluation reports
- **Tags**: free-form key/value annotations (e.g., `dvc_hash`)

## Run Lifecycle

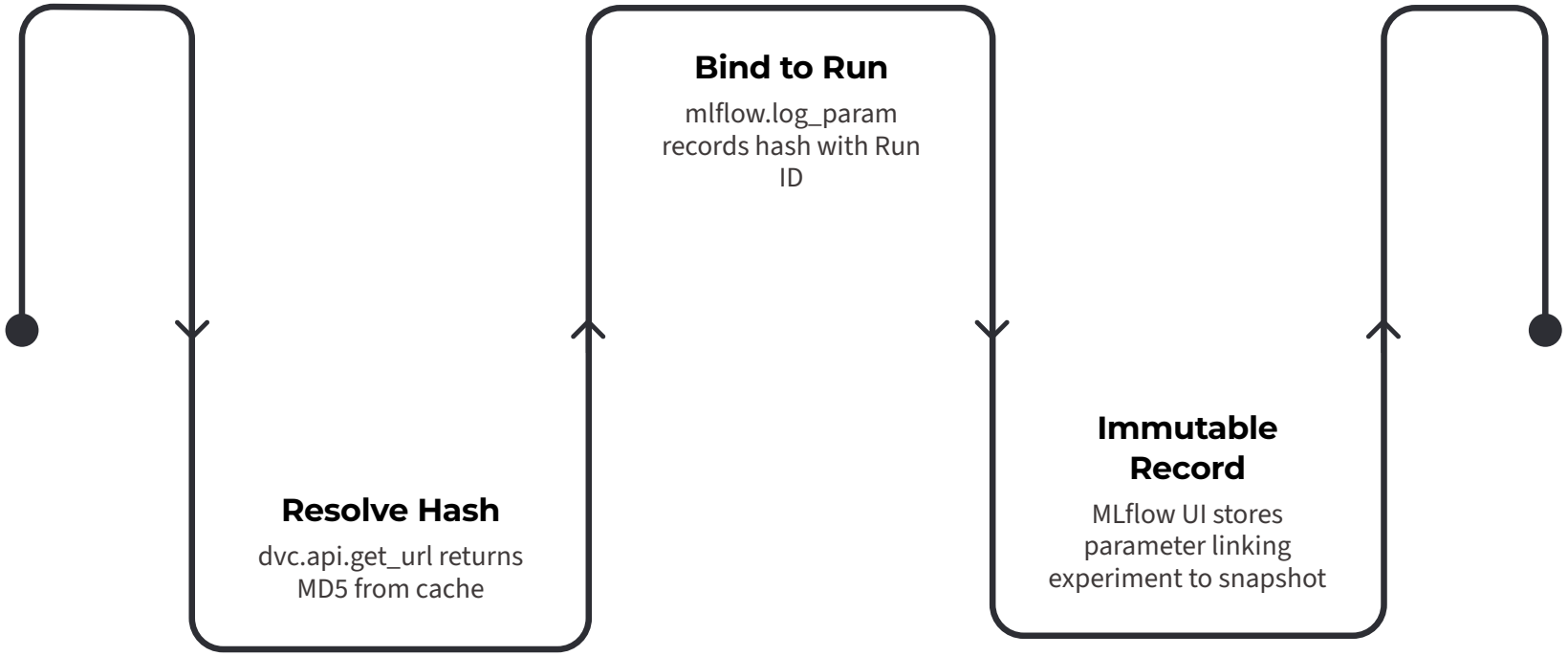
Every Run ID is a globally unique UUID: the primary key linking MLflow records to DVC-managed data states.



Made with GAMMA

# The Integration Logic: The DVC - MLflow Handshake

The integration pivot is a single, deterministic operation: **extract the DVC MD5 hash at runtime and log it as a first-class MLflow parameter**. This binds the experiment record to its exact data snapshot.




One function call at run initialization is sufficient to establish a cryptographically verifiable link between every MLflow Run and its corresponding DVC data state.

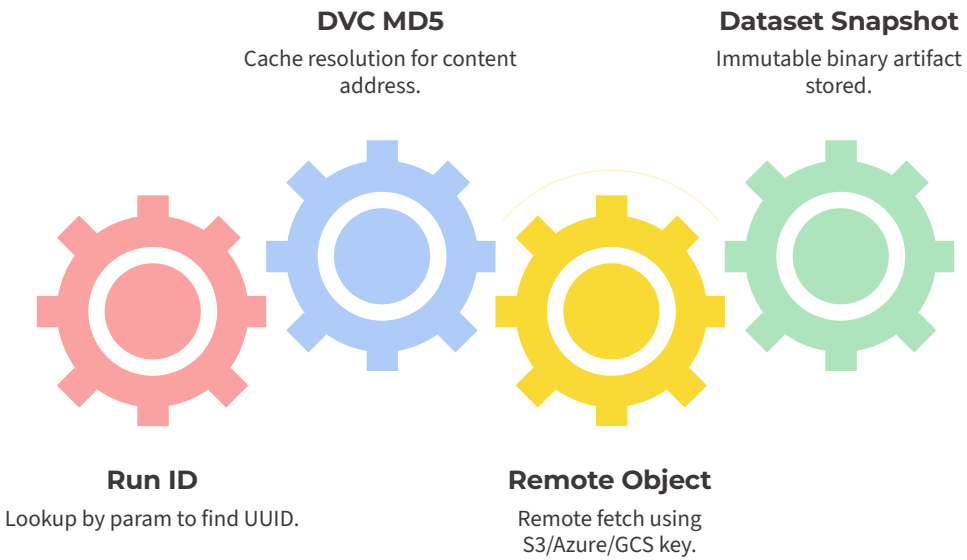
Made with GAMMA

# Achieving Data Lineage: The Traceability Chain

**Lineage** is the capacity to deterministically reconstruct *which data* produced *which model* under *which conditions*. At any point in time, for any run.

### Forward Traceability

Given a Run ID from the MLflow UI, recover the exact dataset: query `dvc_data_hash` parameter → `dvc pull` using that hash.




### Reverse Traceability

Given a dataset version (MD5), identify every MLflow experiment that trained against it, enabling impact analysis when data quality issues surface.

Made with GAMMA

# Model Registry & Governance: Closing the Loop

## DVC-Tracked Model Artifacts

Serialized model files (`.pkl`,`.h5`, `.onnx`, `.pt`) are versioned as DVC artifacts, receiving content-addressable hashes identical to data files. The model binary never enters Git.

```
dvc add models/classifier.pkl
git add models/classifier.pkl.dvc
git commit -m "model: v2.1 trained \
  on dataset a1b2c3"
```

## MLflow Model Registry — Lifecycle Stages

Every registry transition is logged with its associated DVC model hash — enabling complete auditability from inference endpoint back to raw training data.

### None → Staging

Candidate model registered; awaiting validation gate

### Staging → Production

Promoted after passing CI evaluation; DVC hash audited

### Production → Archived

Superseded version frozen; lineage preserved indefinitely

Made with GAMMA

# Workflow Walkthrough: End-to-End Sequence

A complete, reproducible training cycle in four deterministic steps — each step producing a verifiable, immutable artifact.

1

### dvc add data/train.csv

Computes MD5 hash; writes `.dvc` meta file; moves binary to cache. Data is now content-addressed.

2

### git commit -m "data: v3 ingestion"

Commits pointer file to Git history. The repository now encodes a deterministic reference to the dataset snapshot.

3

### python train.py

Calls `dvc.api.get_url()` at initialization; logs `dvc_data_hash` and all hyperparameters via `mlflow.log_param()`; trains model; logs artifacts.

4

### MLflow UI Verification

Navigate to the Run record; confirm `dvc_data_hash` matches `git show HEAD:data/train.csv.dvc`. Lineage is cryptographically verified.

Made with GAMMA